Hazard Analysis
Software Engineering

Team 19, Team BVSH
Shivya Mehta
Anindita Bornomala
Harmanpreet Singh Sagar
Vanessa Lai

Table 1: Revision History

Date	**Developer(s)**	**Change**
Date1	Name(s)	Description of changes
Date2	Name(s)	Description of changes
...	...	...

Contents

1 Introduction **1**

2 Scope and Purpose of Hazard Analysis **1**

3 System Boundaries and Components **2**

4 Critical Assumptions **3**

5 Failure Mode and Effect Analysis **4**

6 Safety and Security Requirements **4**

7 Roadmap **5**

8 Appendix — Generative AI Use Disclosure **6**

8.1 AI Tools Used 6

8.2 How and Where Used 6

8.3 Verification of AI-Generated Content 6

9 Appendix — Reflection **7**

[You are free to modify this template. —SS]

1 Introduction

A hazard is a system condition or failure that could contribute to an undesirable outcome, resulting in losses like inaccurate accreditation assessments, poor deadline recommendations, a loss of course information, or sensitive data breaches.

The Course Deadline Management system is intended to support McMaster University's Computing and Software Department in reviewing Software Engineering course outlines for compliance with applicable university policies and Canadian Engineering Accreditation Board (CEAB) requirements. The system also analyzes course schedules to recommend assignment and midterm deadlines that reduce overlapping student workloads, while leaving final scheduling decisions to instructors.

This document identifies potential hazards associated with the Course Deadline Management system, examines their consequences, and establishes a basis for developing safety and security requirements to mitigate these risks.

2 Scope and Purpose of Hazard Analysis

The purpose of this hazard analysis is to identify potential failures in the Course Deadline Management system that could negatively affect accreditation compliance reviews, course scheduling, or the handling of course information. The analysis covers the processing of course outlines, evaluation against McMaster policies and CEAB requirements, deadline conflict detection, and the generation of scheduling recommendations. The scope is limited to Software Engineering courses within the CAS department and excludes first-year courses, courses outside the department, and the security of the hosting infrastructure itself.

Potential losses resulting from system hazards include:

- Accreditation non-compliance, resulting in unaddressed violations of McMaster policies or CEAB requirements.
- Increased student workload conflicts due to overlapping assignment and midterm deadlines, potentially contributing to academic stress and more MSAF requests.
- Incorrect scheduling decisions resulting from inaccurate or misleading recommendations.

- Loss or corruption of course information, resulting in inaccurate course outline data and scheduling issues.
- Unauthorized disclosure of confidential information, including the master course sheet, compliance findings, and tentative assessment dates.
- Wasted administrator and instructor time due to false compliance findings, incorrectly identified deadline conflicts, or inaccurate recommendations.
- Loss of trust in the system due to repeated inaccurate or unreliable results, potentially leading users to stop using the system and reducing its intended benefits.

3 System Boundaries and Components

The CAS Course Deadline Management system processes Software Engineering course outlines to support accreditation compliance reviews and deadline optimization. Its boundary includes the components responsible for processing course information, evaluating policy and accreditation requirements, detecting scheduling conflicts, generating scheduling suggestions, and managing stored information. The system uses course outlines, a master course sheet for graduate attribute checks, and the academic calendar to identify restricted scheduling periods. The LLM service, hosting infrastructure, users' web browsers, and instructors who prepare course outlines are outside the system boundary, but their interactions with the system and the information they provide are considered in the hazard analysis.

C1 User Interface: Allows CAS administrators and instructors to interact with the system, upload course outlines, and review compliance findings and scheduling recommendations.

C2 Course Information Database: Stores course outline information, extracted assessment details, compliance findings, and tentative assessment dates.

C3 PDF Text Extraction: Extracts text from uploaded PDF course outlines using a text-extraction library.

C4 LLM Information Structuring: Uses an external LLM service to interpret extracted text and structure relevant information, including marking schemes, assignment deadlines, and midterm dates.

C5 Accreditation Compliance Analysis: Evaluates course outline information against applicable McMaster policies and CEAB requirements, including graduate attribute checks using the master course sheet.

C6 Conflict Detection: Analyzes assessment dates across courses to identify overlapping deadlines and potential student workload bottlenecks, taking restricted scheduling periods into account where applicable.

C7 Scheduling Suggestions: Uses a CP-SAT solver to generate proposed assessment dates based on scheduling constraints, identified conflicts, and applicable restricted periods.

C8 Results Presentation: Presents compliance findings, detected conflicts, and proposed scheduling changes to administrators and instructors for review.

C9 Authentication and Access Control: Authenticates users and enforces permissions to restrict access to system functions and confidential information.

C10 Data Export: Exports permitted course information, compliance findings, and scheduling results for authorized users.

C11 External Dependencies: Includes external services and software dependencies, specifically the LLM service used to structure extracted course outline information. The hosting infrastructure and users' web browsers are also external to the system boundary.

4 Critical Assumptions

The hazard analysis is based on the following assumptions about the system's operating environment, users, and input data. These assumptions establish the context for the FMEA, while risks arising when they do not hold will be assessed and appropriate mitigation measures identified.

- **Hosting Infrastructure:** The hosting infrastructure is secured, backed up, and maintained by CAS IT. The security of the hosting infrastructure itself is outside the scope of this analysis.
- **User Accounts:** Users access the system using valid McMaster accounts and do not share their credentials.
- **Master Course Sheet:** The department-provided master course sheet contains correct information for the graduate attribute checks, as assumed in the SRS.
- **Academic Calendar:** The academic calendar dates entered into the system are correct for the term being analyzed.
- **Course Outline Confidentiality:** Course outlines are treated as public documents, consistent with the SRS. Other stored information, including

the master course sheet, compliance findings, and tentative assessment dates, may be confidential.

- **Policy and Accreditation Requirements:** The McMaster policies and CEAB requirements maintained in the system reflect the versions in force for the term being reviewed. Updates are made when applicable requirements change.

5 Failure Mode and Effect Analysis

[The safety requirements in the table do not have to have the prefix SR. The most important thing is to show traceability to your SRS. You might trace to requirements you have already written, or you might need to add new requirements. —SS] [If no safety requirement can be devised, other mitigation strategies can be entered in the table, including strategies involving providing additional documentation, and/or test cases. —SS]

6 Safety and Security Requirements

The requirements below are in addition to safeguards already in the SRS (see Table 4 of the SRS), including user review of extracted data (FR5), flagging of missing fields (FR4), instructor approval of date changes (FR12), login (FR15), role-based access (FR16), and restrictions on data sent to external services (AIR2). Additional safety and security requirements are listed in Table 2. Public outline text may be sent to Copilot, but confidential data and student information shall remain internal (see DR2).

Table 2: Safety and Security Requirements

ID	Hazard(s)	Requirement
SSR1	Outline processing failure resulting in an invalid or incomplete schedule or lost data	The system shall display an actionable error when a course outline PDF cannot be read or the model fails to process it, shall not create assessments for that outline automatically, and shall keep all data already entered.
SSR2	Unsupported or malicious input file	The system shall accept only PDF files smaller than 5 MB, and shall reject any other file before processing it.
SSR3	Assessment scheduled during a restricted period	The system shall use the academic calendar and shall not suggest an assessment date that falls on a holiday, recess, examination period, or test and examination ban period.
SSR4	Out-of-date scheduling suggestion	The system shall mark a date suggestion as out of date if the assessment's date, or the other assessments in the affected weeks, change before the instructor decides on it.
SSR5	Incomplete compliance check or falsely reported absence of findings	The system shall show which compliance rules were checked for each outline, and shall state that a course was not checked when it is missing from the master course sheet, rather than reporting no findings.
SSR6	Unauthorized or untraceable data modification	The system shall record the user, the time, and the previous value whenever an assessment date or extracted value is changed, and shall record each change and retain that audit record for the retention period in DR4.
SSR7	Unauthorized modification of course-rule data	The system shall allow only administrators to change the master course sheet.

7 Roadmap

[Which safety requirements will be implemented as part of the capstone timeline? Which requirements will be implemented in the future? —SS]

8 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

8.1 AI Tools Used

Grok (version 4.7) was used.

8.2 How and Where Used

Grok was used to fix formatting issues and address pull request comments in this Hazard Analysis document, section 6.

8.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

9 Appendix — Reflection

1. What went well while writing this deliverable?

When writing this deliverable, it was relieving for our team to consolidate and organize all the potential risks we thought of on paper. We were able to brainstorm various hazards, as well as critical assumptions thanks to the discussions we've had with stakeholders prior to the deliverable.

2. What pain points did you experience during this deliverable, and how did you resolve them?

A major pain point our team experienced while working on the Hazard Analysis document was considering all the potential risks that could occur before even having a working prototype of our project. With hardware, it is easy to imagine the potential hazards involved with the final product, but software requires a good visualization of what risks can occur from the perspective of the user. Depending on the technological skill level, different users may experience vastly different situations when interacting with our system. However, to resolve this we broke down our system into various components based on the user process (what might they do in chronological order). This helped us to understand where hidden hazards could be located, as well as any assumptions we may have made due to our own expertise with technology.

3. Which of your listed risks had your team thought of before this deliverable, and which did you think of while doing this deliverable? For the latter ones (ones you thought of while doing the Hazard Analysis), how did they come about?

Before the Hazard Analysis deliverable, some potential risk ideas intuitively came to our group while we were brainstorming how to go about developing a prototype of our design, such as potentially leaking sensitive and confidential information (ex. student information). On the other hand, other risks that were only considered while working on the deliverable included submitting malicious file inputs and untraceable data modifications, which we thought of after considering how a user would interact with the system step-by-step. Additionally, discussing with our stakeholders helped us gain different perspectives and better understand what users might be concerned about.

4. Other than the risk of physical harm (some projects may not have any appreciable risks of this form), list at least 2 other types of risk in software products. Why are they important to consider?

One type of risk in software products is cybersecurity risks and potential data leaks. In our project, while we aren't working with critically sensitive information, we need to consider any administrative or personal information, as well

as ensuring that authorized users are able to interact with the system without giving access to non-authorized users. This is important because data breaches can cause harm by exposing personal information to people who may have malicious intentions without consent.

Another type of software risk is misinformation and/or component failures when a user is interacting with the system. Not only is this frustrating for the user, but this can result in the failure of the complete product. One of the first things our team recognized with our project was that the last thing our users would want is to navigate a software platform that has a high learning curve or false information, as they are not expected to have as high technical knowledge as we do. Therefore, designing a system that is both simple and provides accurate data is essential to the success of our project.